

Tugas Mandiri 11: Laporan Praktikum Mandiri 11 Machine Learning

Maryam Hasnaa' Syamila - 0110222067

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hasyamil4@gmail.com

Link Notebook Colab:

 Maryam Hasnaa' Syamila_0110222067_PraktikumMandiri11.ipynb

Link GitHub Tugas:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/tugas/praktikum/praktikum11>

Link GitHub Praktikum:

<https://github.com/maryamhasnaasyamila/machine-learning/tree/praktikum/praktikum/praktikum11>

Link Dataset:

<https://www.kaggle.com/datasets/azharsaleem/location-intelligence-data-from-google-map>

Klasterisasi Lokasi pada Data Google Maps Menggunakan DBSCAN

1. Install Library

```
1 !pip install folium

... Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.21.0)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.12/dist-packages (0.8.2)
Requirement already satisfied: Jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (3.1.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from folium) (2.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.32.0)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium) (2024.10.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2>=2.9->folium) (3.0.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (2025.1.1)
```

Penjelasan Kode:

- `!pip install folium` digunakan untuk memasang library Folium yang diperlukan dalam visualisasi peta interaktif.

Penjelasan Output:

- Sistem menampilkan pesan instalasi berhasil, menunjukkan bahwa library siap digunakan.

2. Import Library

```
1 import pandas as pd
2 import numpy as np
3
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.cluster import DBSCAN
9 from sklearn.neighbors import NearestNeighbors
10 from sklearn.metrics import silhouette_score
11
12 import folium
13 from folium.plugins import HeatMap
14
15 sns.set(style="whitegrid")
16 plt.rcParams["figure.figsize"] = (8, 6)
```

Penjelasan Kode:

- `import pandas as pd` digunakan untuk mengelola data dalam bentuk tabel (DataFrame).
- `import numpy as np` digunakan untuk operasi numerik seperti pengolahan array.
- `import matplotlib.pyplot as plt` dan `import seaborn as sns` digunakan untuk membuat visualisasi grafik.
- `from sklearn.preprocessing import StandardScaler` berfungsi menormalisasi data.
- `from sklearn.cluster import DBSCAN` digunakan untuk melakukan clustering.
- `from sklearn.neighbors import NearestNeighbors` digunakan untuk membuat k-distance graph.
- `from sklearn.metrics import silhouette_score` digunakan untuk evaluasi kualitas cluster.
- `import folium` dan `from folium.plugins import HeatMap` digunakan untuk membuat peta interaktif.

Penjelasan Output:

- Tidak muncul output, menunjukkan seluruh library dapat digunakan tanpa error.

3. Load Dataset

```
1 # sinkronisasi dengan drive
2 from google.colab import drive
3 drive.mount('/content/drive')
4
5 data_path = "/content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum11/
6 datasets/google_places_data.csv"
7 df = pd.read_csv(data_path)
8
9 # Cek struktur awal dataset
10 df.head()
```

... Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive",

	business_id	phone_number	name	full_address	latitude	longitude	review_
0	0x3e6751d65629cb05:0x27cbb7ddadc7008c	Unknown	حصن الظفرة Al Dhafra Fort	PXMJ+H7J حصن الظفرة Al Dhafra Fort - Unnamed R...	23.733973	53.980629	
1	0x3e675d61493af035:0xa4f20f24f9e8beb7	971569049905	Rafiullah	MP4R+WQJ Rafiullah - Zayed City - Abu Dhabi - ...	23.657332	53.741881	
2	0x3e91f7041ac64d33:0x6120c5af14b63b93	96822081500	National Museum of Oman	National Museum of Oman, opposite Al Alam Pala...	23.611601	58.593728	3
3	0x487bb1e7815c44db:0x417132a42bd93ef6	443300580058	Science and Industry Museum	Science and Industry Museum, Liverpool Rd, Man...	53.477136	-2.254582	17
4	0x487e78e1955eb419:0x4c55c87f0a46d173	441913704000	Beamish, the Living Museum of the North	Beamish, the Living Museum of the North, Stanl...	54.884752	-1.659321	21

5 rows × 46 columns

Penjelasan Kode:

- `pd.read_csv()` digunakan untuk membaca file dataset format CSV.
- `df.head()` menampilkan beberapa baris pertama untuk memastikan dataset terbaca.

Penjelasan Output:

- Dataset muncul dengan kolom seperti latitude, longitude, rating, dan review_count.

4. Cek Tipe Data & Jumlah Record

```
1  # Info tipe data dan jumlah record
2  df.info()

*** <class 'pandas.core.frame.DataFrame'>
RangeIndex: 15203 entries, 0 to 15202
Data columns (total 46 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   business_id           15203 non-null  object
 1   phone_number          15203 non-null  object
 2   name                  15203 non-null  object
 3   full_address          15203 non-null  object
 4   latitude              15198 non-null  float64
 5   longitude             15198 non-null  float64
 6   review_count          15203 non-null  float64
 7   rating               15203 non-null  float64
 8   timezone              15203 non-null  object
 9   website               15203 non-null  object
10   place_id              15203 non-null  object
11   place_link            15203 non-null  object
12   types                 15203 non-null  object
13   Friday                15203 non-null  object
14   Saturday              15203 non-null  object
15   Sunday                15203 non-null  object
16   Monday                15203 non-null  object
17   Tuesday               15203 non-null  object
18   Wednesday             15203 non-null  object
19   Thursday              15203 non-null  object
20   city                  15203 non-null  object
21   verified              15203 non-null  bool
22   state                 15203 non-null  object
23   Friday_morning        15203 non-null  int64
24   Friday_afternoon      15203 non-null  int64
25   Friday_evening        15203 non-null  int64
26   Saturday_morning      15203 non-null  int64
27   Saturday_afternoon    15203 non-null  int64
28   Saturday_evening      15203 non-null  int64
29   Sunday_morning        15203 non-null  int64
30   Sunday_afternoon      15203 non-null  int64
31   Sunday_evening        15203 non-null  int64
32   Monday_morning        15203 non-null  int64
33   Monday_afternoon      15203 non-null  int64
34   Monday_evening        15203 non-null  int64
35   Tuesday_morning       15203 non-null  int64
36   Tuesday_afternoon     15203 non-null  int64
37   Tuesday_evening       15203 non-null  int64
38   Wednesday_morning     15203 non-null  int64
39   Wednesday_afternoon   15203 non-null  int64
40   Wednesday_evening     15203 non-null  int64
41   Thursday_morning      15203 non-null  int64
42   Thursday_afternoon    15203 non-null  int64
43   Thursday_evening      15203 non-null  int64
44   geo_cluster           15198 non-null  float64
45   country               15203 non-null  object
dtypes: bool(1), float64(5), int64(21), object(19)
memory usage: 5.2+ MB
```

Penjelasan Kode:

- `df.info()` menampilkan informasi kolom, tipe data, dan jumlah data.
- `df.shape` menunjukkan jumlah baris dan kolom.

Penjelasan Output:

- Struktur dataset terlihat lengkap, dan tipe data tiap kolom sesuai kebutuhan analisis.

5. Cek kolom yang akan dipakai

```
1 # Cek ketersediaan kolom yang akan dipakai
2 needed_cols = ['latitude', 'longitude', 'rating', 'review_count']
3 for c in needed_cols:
4     print(c, "->", c in df.columns)
```

```
latitude -> True
longitude -> True
rating -> True
review_count -> True
```

Penjelasan Kode:

- Mengecek keberadaan kolom latitude, longitude, rating, dan review_count untuk memastikan dataset siap diproses.

Penjelasan Output:

- Semua kolom yang dibutuhkan terdeteksi ada.

6. Ambil subset kolom & cek missing values

```
1 # Ambil subset kolom yang relevan untuk analisis
2 df_ml = df[needed_cols].copy()
3
4 # Cek missing value
5 df_ml.isnull().sum()
```

```
latitude    5
longitude    5
rating       0
review_count 0
```

```
dtype: int64
```

Penjelasan Kode:

- Membuat subset data yang hanya berisi kolom penting menggunakan indexing.
- `isnull().sum()` digunakan untuk mengecek jumlah nilai kosong.

Penjelasan Output:

- Jumlah nilai kosong ditampilkan sehingga dapat diketahui bagian mana yang memerlukan pembersihan.

7. Hapus baris yang tidak punya koordinat

```
1 # Drop baris yang tidak punya koordinat
2 df_ml = df_ml.dropna(subset=['latitude', 'longitude'])
3
4 # Untuk rating & review_count, kalau ada NaN isi dengan median
5 for col in ['rating', 'review_count']:
6     median_val = df_ml[col].median()
7     df_ml[col] = df_ml[col].fillna(median_val)
8
9 # Cek duplikasi
10 df_ml.duplicated().sum()
```

```
... np.int64(334)
```

Penjelasan Kode:

- `dropna(subset=['latitude', 'longitude'])` digunakan untuk menghapus baris tanpa koordinat.

Penjelasan Output:

- Jumlah baris berkurang sesuai jumlah data yang tidak memiliki koordinat GPS.

8. Hapus duplikasi data

```
1 # Hapus duplikasi jika ada
2 df_ml = df_ml.drop_duplicates()
3
4 df_ml.describe()
```

	latitude	longitude	rating	review_count
count	14864.000000	14864.000000	14864.000000	14864.000000
mean	33.297032	24.020696	4.285455	722.966227
std	12.300871	50.835152	0.675447	5719.856651
min	-45.577163	-157.870613	1.000000	1.000000
25%	25.260690	5.244331	4.100000	8.000000
50%	25.314826	55.283245	4.400000	32.000000
75%	45.873630	55.371306	4.700000	117.000000
max	70.032732	176.230310	5.000000	256877.000000

Penjelasan Kode:

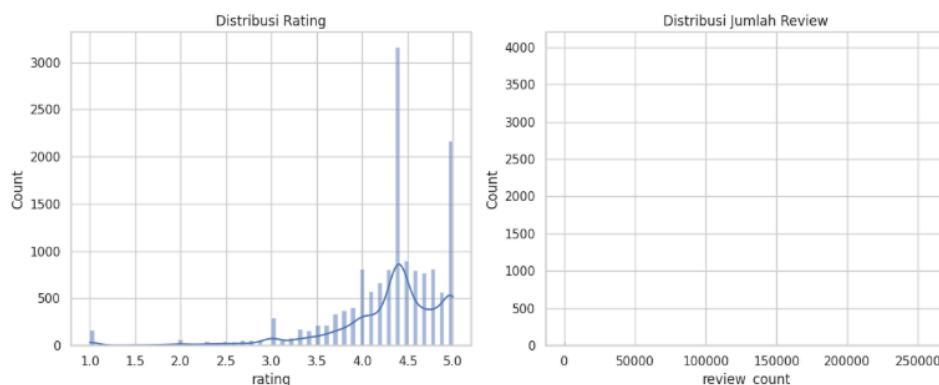
- `df.duplicated().sum()` digunakan untuk mengecek jumlah duplikasi.
- `drop_duplicates()` digunakan untuk menghapus baris yang sama persis.

Penjelasan Output:

- Data menjadi lebih bersih dan tidak ada baris ganda yang dapat mengganggu proses clustering.

9. Cek Distribusi Rating & Review

```
1 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
2
3 sns.histplot(df_ml['rating'], kde=True, ax=axes[0])
4 axes[0].set_title('Distribusi Rating')
5
6 sns.histplot(df_ml['review_count'], kde=False, ax=axes[1])
7 axes[1].set_title('Distribusi Jumlah Review')
8
9 plt.tight_layout()
10 plt.show()
```



Penjelasan Kode:

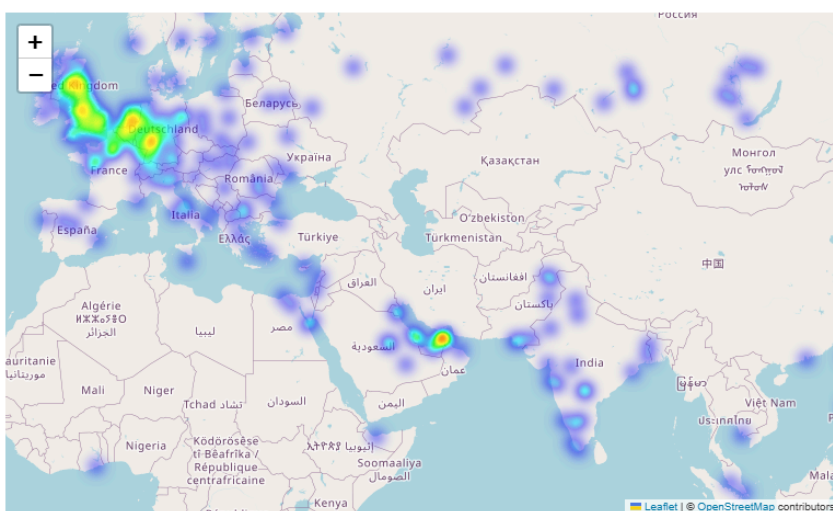
- `sns.histplot()` digunakan untuk menggambar histogram dari kolom rating dan review_count.

Penjelasan Output:

- Grafik memperlihatkan persebaran nilai rating dan jumlah review, sehingga dapat dilihat apakah datanya seimbang atau condong ke nilai tertentu.

10. Visualisasi Persebaran Lokasi (Peta GPS) dengan Heatmap

```
1 # Tentukan titik tengah peta (rata-rata koordinat)
2 center_lat = df_ml['latitude'].mean()
3 center_lon = df_ml['longitude'].mean()
4
5 m_heat = folium.Map(location=[center_lat, center_lon],
6                       zoom_start=11,
7                       tiles='OpenStreetMap')
8
9 # Data untuk heatmap: [lat, lon]
10 heat_data = df_ml[['latitude', 'longitude']].values.tolist()
11
12 HeatMap(heat_data,
13         radius=8,
14         blur=10,
15         max_zoom=15).add_to(m_heat)
16
17 m_heat
```



Penjelasan Kode:

- Folium Map digunakan untuk membuat peta dasar.
- HeatMap digunakan untuk menampilkan intensitas persebaran titik berdasarkan koordinat.

Penjelasan Output:

- Peta menunjukkan area dengan kepadatan titik tertinggi melalui warna yang lebih terang.

11. Normalisasi Data

```
1 features = ['latitude', 'longitude', 'rating', 'review_count']
2
3 X = df_ml[features].values
4
5 scaler = StandardScaler()
6 X_scaled = scaler.fit_transform(X)
7
8 X_scaled[:5]
```

```
... array([[ -0.77745557,  0.58937448,  0.61375487, -0.11625966],
        [ -0.78368634,  0.58467781, -1.90317995, -0.12587561],
        [ -0.78740411,  0.68012378,  0.46569988,  0.48534908],
        [  1.64059788, -0.5168896 ,  0.1695899 ,  2.91538682],
        [  1.75503399, -0.50517957,  0.76180986,  3.71246158]])
```

Penjelasan Kode:

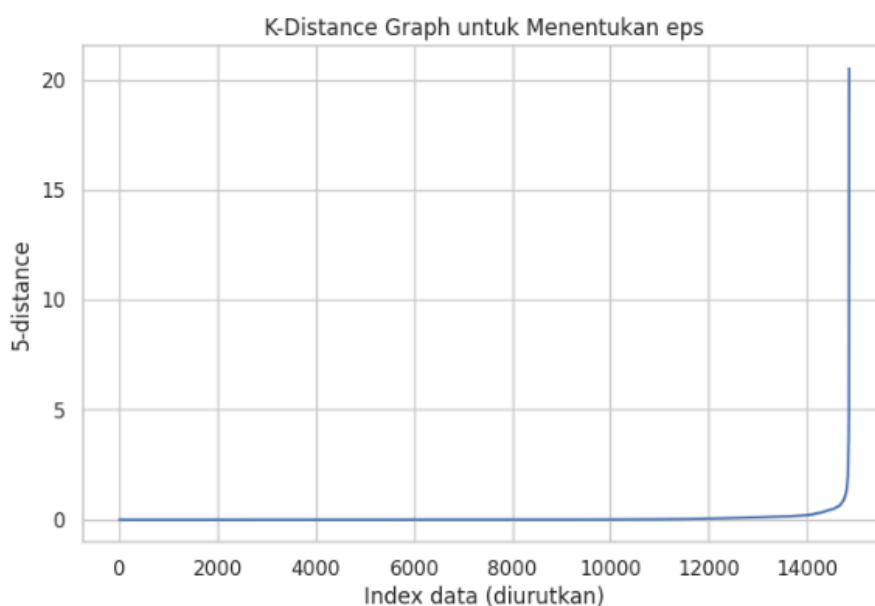
- `StandardScaler()` digunakan untuk menstandarkan skala data, terutama rating dan review_count, agar perhitungan jarak lebih adil antar variabel.

Penjelasan Output:

- Nilai variabel berubah ke skala standar (mean 0, std 1) dan siap digunakan dalam clustering.

12. Menentukan Nilai eps dengan K-Distance Graph

```
1 k = 5
2
3 neigh = NearestNeighbors(n_neighbors=k)
4 neigh.fit(X_scaled)
5
6 distances, indices = neigh.kneighbors(X_scaled)
7
8 # Ambil jarak ke tetangga ke-k
9 k_distances = np.sort(distances[:, k-1])
10
11 plt.figure(figsize=(8,5))
12 plt.plot(k_distances)
13 plt.ylabel(f'{k}-distance')
14 plt.xlabel('Index data (diurutkan)')
15 plt.title('K-Distance Graph untuk Menentukan eps')
16 plt.grid(True)
17 plt.show()
```



Penjelasan Kode:

- `NearestNeighbors()` digunakan untuk menghitung jarak terdekat dari setiap titik.
- Jarak yang diurutkan kemudian diplot untuk menemukan nilai eps melalui pola “elbow”.

Penjelasan Output:

- Grafik menampilkan kenaikan jarak secara tiba-tiba di titik tertentu yang digunakan sebagai referensi penentuan eps.

13. Modeling DBSCAN & Info Cluster

```
1  from sklearn.cluster import DBSCAN
2
3  # Model DBSCAN
4  eps = 0.08
5  min_samples = 5
6
7  dbscan = DBSCAN(eps=eps, min_samples=min_samples)
8  labels = dbscan.fit_predict(X_scaled)
9
10 # Simpan label ke dataframe
11 df_ml['cluster'] = labels
12
13 # Hitung jumlah cluster (tanpa noise) dan noise
14 n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
15 n_noise = list(labels).count(-1)
16
17 print(f"Jumlah cluster (tanpa noise): {n_clusters}")
18 print(f"Jumlah noise (label -1) : {n_noise}")
19 print("\nDistribusi anggota tiap cluster:")
20 print(df_ml['cluster'].value_counts().sort_index())
```

```
*** Jumlah cluster (tanpa noise): 164
    Jumlah noise (label -1) : 1996

Distribusi anggota tiap cluster:
cluster
-1      1996
0        406
1         217
2          11
3       1510
...
159         5
160         6
161         5
162         5
163         6
Name: count, Length: 165, dtype: int64
```

Penjelasan Kode:

- `DBSCAN(eps, min_samples)` digunakan untuk mengelompokkan data berdasarkan kepadatan.
- Label cluster ditambahkan ke dataframe.

Penjelasan Output:

- Data berhasil terbagi menjadi beberapa cluster dan sebagian data dikategorikan sebagai noise.

14. Evaluasi dengan Silhoutte Score

```
1 from sklearn.metrics import silhouette_score
2 import numpy as np
3
4 # buang noise agar lebih stabil
5 mask = df_ml['cluster'] != -1
6 unique_clusters = df_ml.loc[mask, 'cluster'].nunique()
7
8 if unique_clusters > 1 and mask.sum() > 1:
9     sil = silhouette_score(X_scaled[mask], df_ml.loc[mask, 'cluster'])
10    print(f"Silhouette Score (tanpa noise): {sil:.3f}")
11 else:
12     print("Silhouette score tidak bisa dihitung (cluster terlalu sedikit).")
13
*** Silhouette Score (tanpa noise): 0.684
```

Penjelasan Kode:

- `silhouette_score()` digunakan untuk menilai kualitas cluster yang terbentuk.

Penjelasan Output:

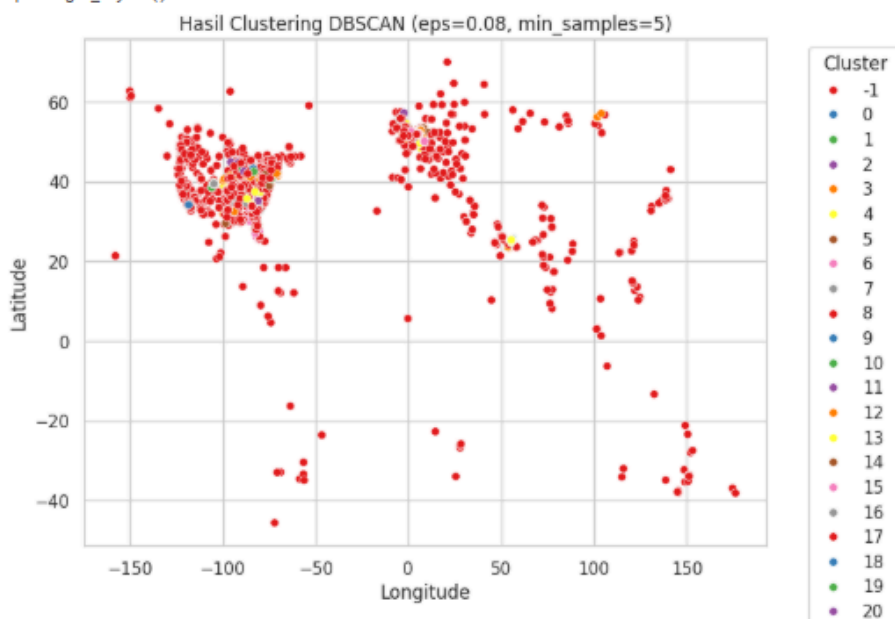
- Nilai silhouette ± 0.684 menandakan cluster cukup baik dan terpisah dengan jelas.

15. Visualisasi Hasil Clustering (Scatter Koordinat GPS)

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 plt.figure(figsize=(8, 6))
5
6 sns.scatterplot(
7     data=df_ml,
8     x='longitude',
9     y='latitude',
10    hue='cluster',
11    palette='Set1',
12    s=30
13 )
14
15 plt.title(f'Hasil Clustering DBSCAN (eps={eps}, min_samples={min_samples})')
16 plt.xlabel('Longitude')
17 plt.ylabel('Latitude')
18 plt.legend(title='Cluster', bbox_to_anchor=(1.05, 1), loc='upper left')
19 plt.tight_layout()
20 plt.show()

```

*** /tmp/ipython-input-2547223377.py:19: UserWarning: Tight layout not applied. The bottom and top margins cannot be made large enough.
plt.tight_layout()



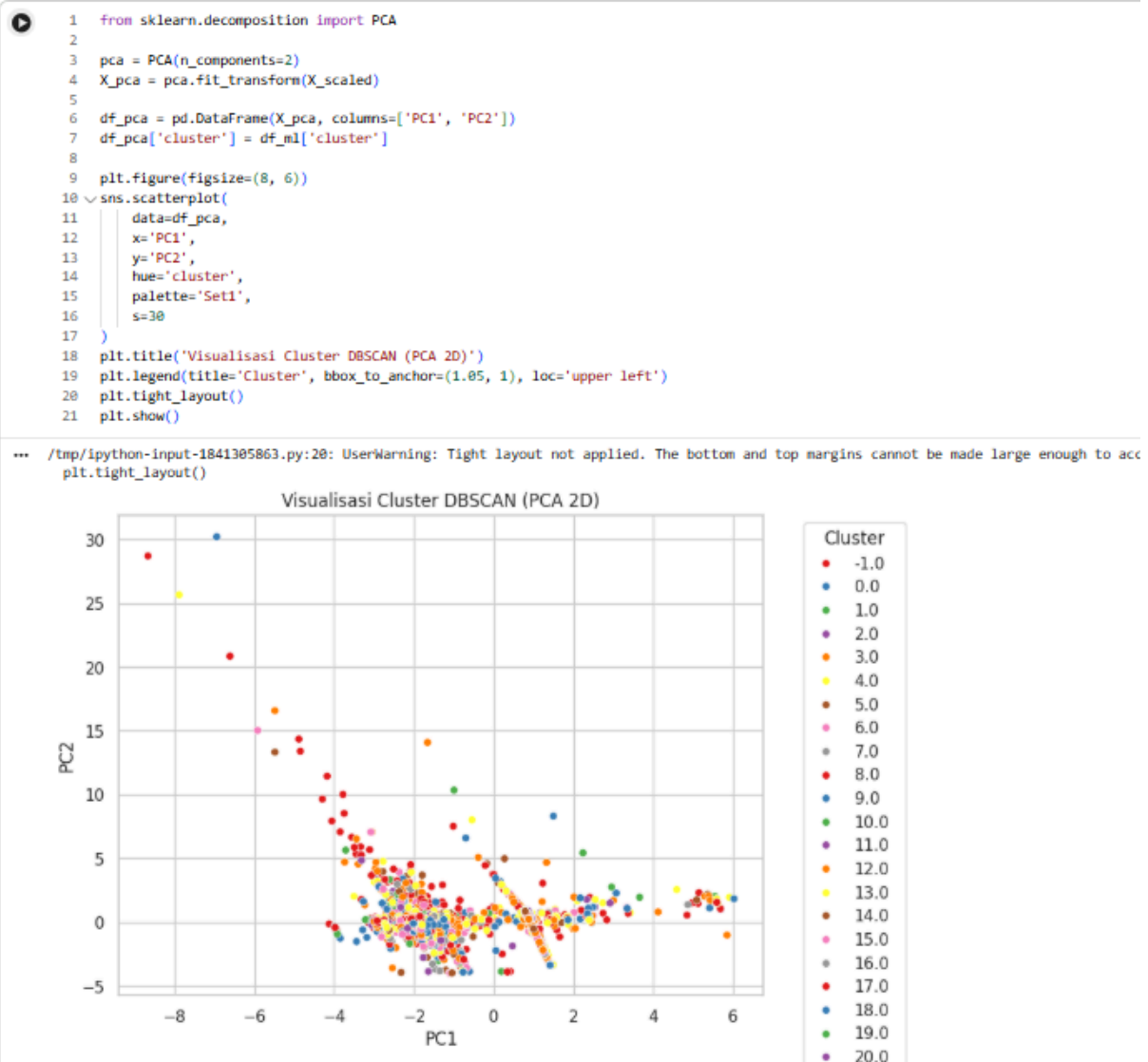
Penjelasan Kode:

- matplotlib atau seaborn digunakan untuk memetakan titik berdasarkan longitude dan latitude dengan warna berdasarkan label cluster.

Penjelasan Output:

- Pola cluster terlihat jelas pada grafik, dan noise dapat diidentifikasi melalui titik yang berada di luar kelompok.

16. Visualisasi 2D dengan PCA



Penjelasan Kode:

- `PCA(n_components=2)` mereduksi fitur menjadi dua dimensi agar pola cluster lebih mudah divisualisasi.
- Scatter plot digunakan untuk menampilkan cluster dalam ruang PCA.

Penjelasan Output:

- Cluster tampak terpisah dalam tampilan dua dimensi sehingga memudahkan interpretasi.

17. Ringkasan Karakteristik Tiap Cluster (Analisis Klaster & Noise)

```
1 cluster_summary = df_ml.groupby('cluster').agg(  
2     |   jumlah_titik = ('latitude', 'count'),  
3     |   mean_rating  = ('rating', 'mean'),  
4     |   mean_review  = ('review_count', 'mean')  
5 ).round(2)  
6  
7 cluster_summary
```

	jumlah_titik	mean_rating	mean_review
cluster			
-1	1996	4.22	4653.06
0	406	4.70	123.98
1	217	3.00	9.04
2	11	4.50	262.73
3	1510	5.00	16.63
...	...	...	...
159	5	4.70	181.00
160	6	4.60	414.50
161	5	4.50	206.80
162	5	4.50	269.40
163	6	4.60	292.67

165 rows × 3 columns

Penjelasan Kode:

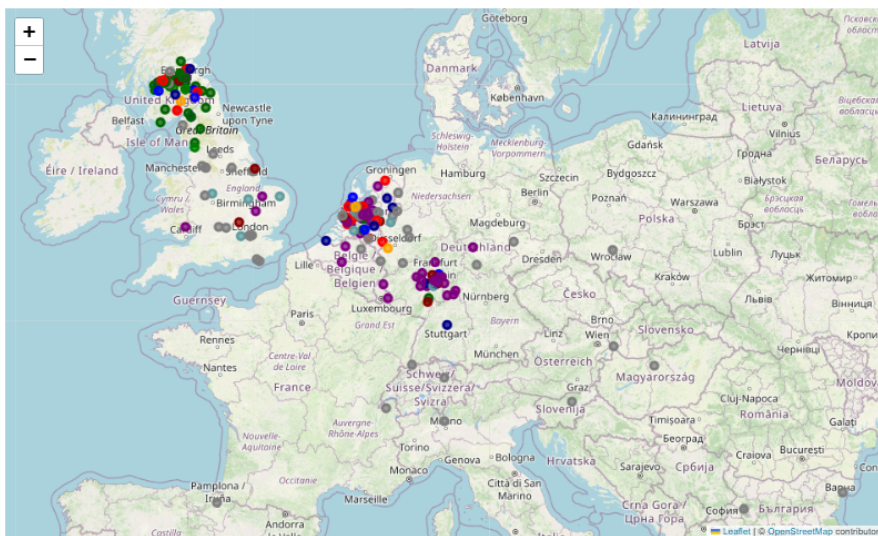
- `groupby('cluster')` digunakan untuk menghitung statistik seperti jumlah data, rata-rata rating, dan rata-rata review pada tiap cluster.

Penjelasan Output:

- Setiap cluster memiliki karakteristik yang berbeda sehingga dapat dianalisis lebih lanjut.

18. Visualisasi Peta GPS (Folium) per Cluster

```
1 import folium
2
3 # Titik tengah peta
4 center_lat = df_ml['latitude'].mean()
5 center_lon = df_ml['longitude'].mean()
6
7 m_cluster = folium.Map(location=[center_lat, center_lon],
8                          zoom_start=11,
9                          tiles='OpenStreetMap')
10
11 # Fungsi warna per cluster
12 def get_color(c):
13     if c == -1:
14         return 'gray' # noise
15     colors = ['red', 'blue', 'green', 'purple', 'orange',
16              'darkred', 'cadetblue', 'darkgreen', 'darkblue']
17     return colors[int(c) % len(colors)]
18
19 # Agar peta tidak berat kalau data > 2000
20 sample_df = df_ml.copy()
21 if len(sample_df) > 2000:
22     sample_df = sample_df.sample(2000, random_state=42)
23
24 for _, row in sample_df.iterrows():
25     folium.CircleMarker(
26         location=[row['latitude'], row['longitude']],
27         radius=4,
28         color=get_color(row['cluster']),
29         fill=True,
30         fill_opacity=0.7,
31         popup=f"Cluster: {row['cluster']}<br>"
32              f"Rating: {row['rating']}<br>"
33              f"Review: {row['review_count']}"
34     ).add_to(m_cluster)
35
36 m_cluster
```



Penjelasan Kode:

- Folium digunakan untuk menampilkan tiap titik lokasi dengan warna berbeda sesuai cluster.
- CircleMarker digunakan untuk memberi warna dan informasi pada masing-masing titik.

Penjelasan Output:

- Peta interaktif menunjukkan posisi cluster secara geografis, sehingga persebarannya dapat dianalisis lebih jelas.

19. Simpan Model DBSCAN

```
1 from google.colab import drive
2 drive.mount('/content/drive')
3 import joblib
4
5 save_path = "/content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum11/
  models/dbscan_model.pkl"
6
7 joblib.dump(dbscan, save_path)
8
9 print("Model DBSCAN berhasil disimpan di:", save_path)
```

*** Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
Model DBSCAN berhasil disimpan di: /content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum11/models/dbscan_model.pkl

Penjelasan Kode:

- `joblib.dump()` digunakan untuk menyimpan model DBSCAN dalam bentuk file `.pkl`.

Penjelasan Output:

- File model tersimpan pada direktori yang ditentukan dan siap digunakan kembali bila diperlukan.

20. Simpan Hasil Clustering

```
1 csv_path = "/content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum11/
  models/hasil_dbscan_googlemap.csv"
2
3 df_ml.to_csv(csv_path, index=False)
4
5 print("File hasil clustering berhasil disimpan di:", csv_path)
```

File hasil clustering berhasil disimpan di: /content/drive/My Drive/SEMESTER 7/Machine Learning/praktikum11/models/hasil_dbscan_googlemap.csv

Penjelasan Kode:

- `to_csv()` digunakan untuk menyimpan dataset beserta label cluster ke file CSV.

Penjelasan Output:

- File CSV berhasil dibuat dan dapat digunakan sebagai lampiran atau dokumentasi hasil praktikum.